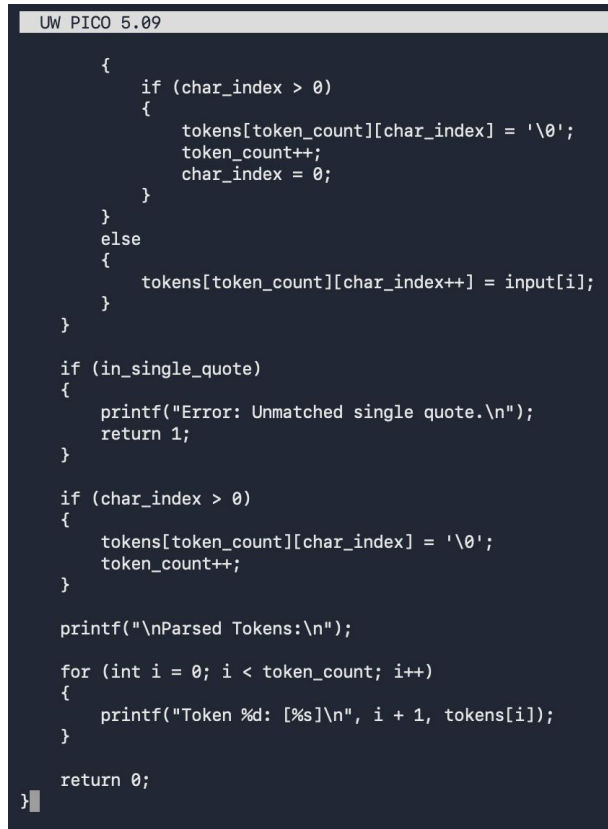


SKILL_WEEK-05

Task 1: Apply Single Quotes to preserve literal content, ignore variable expansion, store quoted strings, validate parsing results, and test edge cases.



```
UW PICO 5.09

{
    if (char_index > 0)
    {
        tokens[token_count][char_index] = '\\0';
        token_count++;
        char_index = 0;
    }
}
else
{
    tokens[token_count][char_index++] = input[i];
}
}

if (in_single_quote)
{
    printf("Error: Unmatched single quote.\\n");
    return 1;
}

if (char_index > 0)
{
    tokens[token_count][char_index] = '\\0';
    token_count++;
}

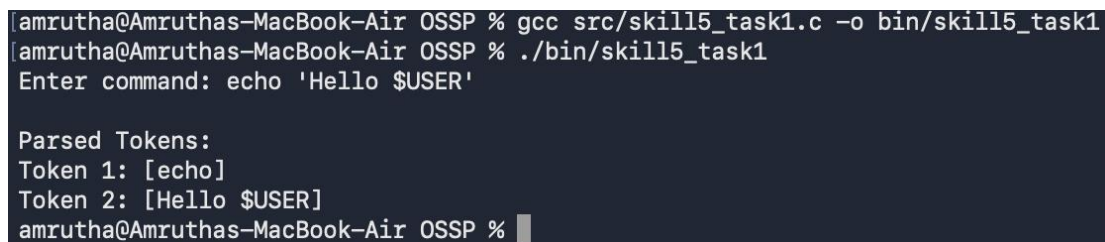
printf("\\nParsed Tokens:\\n");

for (int i = 0; i < token_count; i++)
{
    printf("Token %d: [%s]\\n", i + 1, tokens[i]);
}

return 0;
}
```

Fig 1: C program for parsing and preserving literal content within single quotes

Test case - 1:



```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill5_task1.c -o bin/skill5_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task1
Enter command: echo 'Hello $USER'

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello $USER]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 2: Parsing a single-quoted string while preserving literal content and ignoring variable expansion

Test case - 2:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill15_task1
Enter command: echo 'Hello World From OSSP'

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello World From OSSP]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 3: Preserving spaces inside a single-quoted string during parsing

Test case - 3:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill15_task1
Enter command: echo 'Hello World
Error: Unmatched single quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 4: Detection of an unmatched single quote during command parsing



Fig 5: Flowchart for parsing and validating single-quoted strings

Result:

Single-quoted strings were parsed successfully while preserving spaces and literal content. Variable expansion was ignored, quoted strings were stored as single tokens, and unmatched-quote edge cases were detected correctly.

Task 2: Apply Double Quotes to preserve spaces, allow variable expansion, parse nested tokens, validate outputs, and test quoted commands.

```
UW PICO 5.09

printf("Enter command: ");
fgets(input, sizeof(input), stdin);

input[strcspn(input, "\n")] = '\0';

for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '"')
    {
        in_double_quote = !in_double_quote;
    }
    else if (isspace((unsigned char)input[i]) && !in_double_quote)
    {
        if (char_index > 0)
        {
            tokens[token_count][char_index] = '\0';
            token_count++;
            char_index = 0;
        }
    }
    else
    {
        tokens[token_count][char_index++] = input[i];
    }
}

if (in_double_quote)
{
    printf("Error: Unmatched double quote.\n");
    return 1;
}

if (char_index > 0)
{
    tokens[token_count][char_index] = '\0';
    token_count++;
}

printf("\nParsed Tokens:\n");

for (int i = 0; i < token_count; i++)
{
    char expanded[MAX_LENGTH];

    expand_variable(tokens[i], expanded);

    printf("Token %d: [%s]\n", i + 1, expanded);
}

return 0;
}
```

Fig 6: C program for parsing double-quoted strings with variable expansion

Test Case - 1:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill5_task2.c -o bin/skill5_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task2
Enter command: echo "Hello $USER"

Parsed Tokens:
Token 1: [echo]
Token 2: [Hello amrutha]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 7: Parsing a double-quoted string with variable expansion

Test case - 2:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task2
Enter command: echo "Operating Systems Skill Five"

Parsed Tokens:
Token 1: [echo]
Token 2: [Operating Systems Skill Five]
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 8: Preserving spaces inside a double-quoted string

Test case - 3:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill5_task2
Enter command: echo "Hello World
Error: Unmatched double quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 9: Detection of an unmatched double quote during parsing

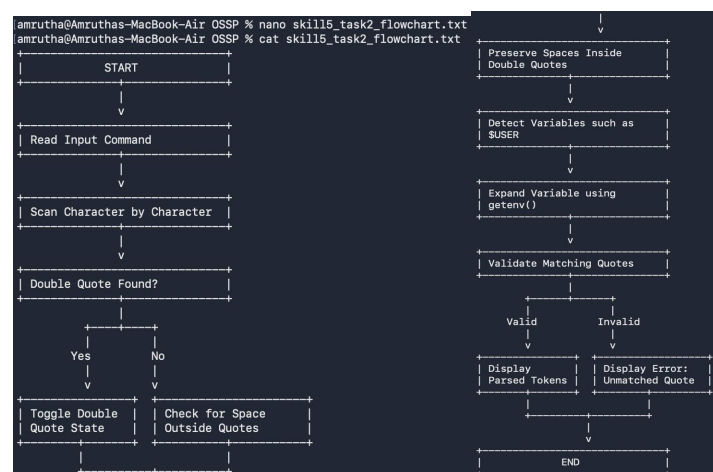


Fig 10: Flowchart for parsing double-quoted strings with variable expansion

Result:

Double-quoted strings were parsed successfully while preserving spaces and allowing variable expansion. Environment variables such as \$USER were expanded correctly, and unmatched double-quote cases were detected successfully.